# Apache Kafka: Module 1

A comprehensive introduction to messaging systems, Kafka's core architecture, and hands-on practice with your first Kafka cluster.

BEGINNER

90 MIN THEORY · 60 MIN EXERCISES

# Module 1 — Learning Objectives

By the end of this module, you will be able to reason clearly about Kafka's architecture and run your first local cluster.

01

## Define a Messaging System

Explain what a message broker does and distinguish between point-to-point (queue) and publish/subscribe patterns.

02

## Explain Why Kafka Was Created

Describe the real problem LinkedIn faced in 2010 and why traditional systems could not solve it.

03

## Identify Kafka's Core Components

Define topics, partitions, offsets, producers, consumers, brokers, and clusters — and draw the relationships between them.

04

## Describe the Data Ecosystem

Explain how Kafka acts as a central data hub connecting multiple source and target systems across an organization.

05

## Trace the Full Kafka Workflow

Follow a message from production through storage and consumption, step by step across all 8 lifecycle stages.

06

## Run Your First Kafka Environment

Start a local Kafka broker using KRaft mode and produce/consume your first messages using command-line tools.

## Prerequisites

- Java 17+ installed on your machine
- Terminal / command-line familiarity
- Basic understanding of client-server architecture

## Module At a Glance

**Duration:** 90 min theory + 60 min exercises = 2.5 hours total

**Difficulty:** Beginner — no prior Kafka knowledge required

**Tools:** Apache Kafka 4.2.0 (KRaft mode) or Docker

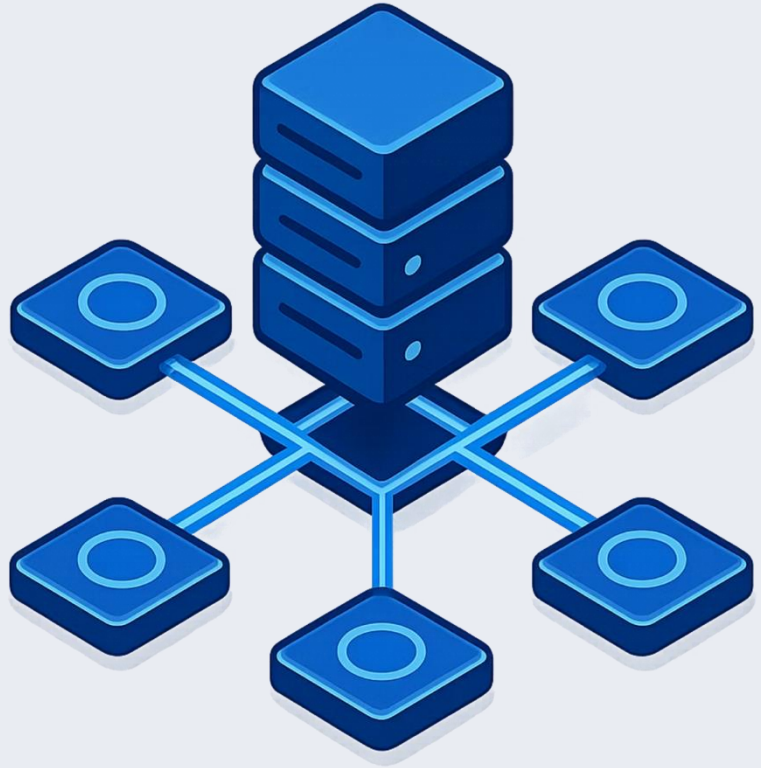
**Primary Sources:** *Kafka: The Definitive Guide* (O'Reilly, 2017) · Apache Kafka Official Docs v4.2.0

SECTION 1.1

# What Is a Messaging System?

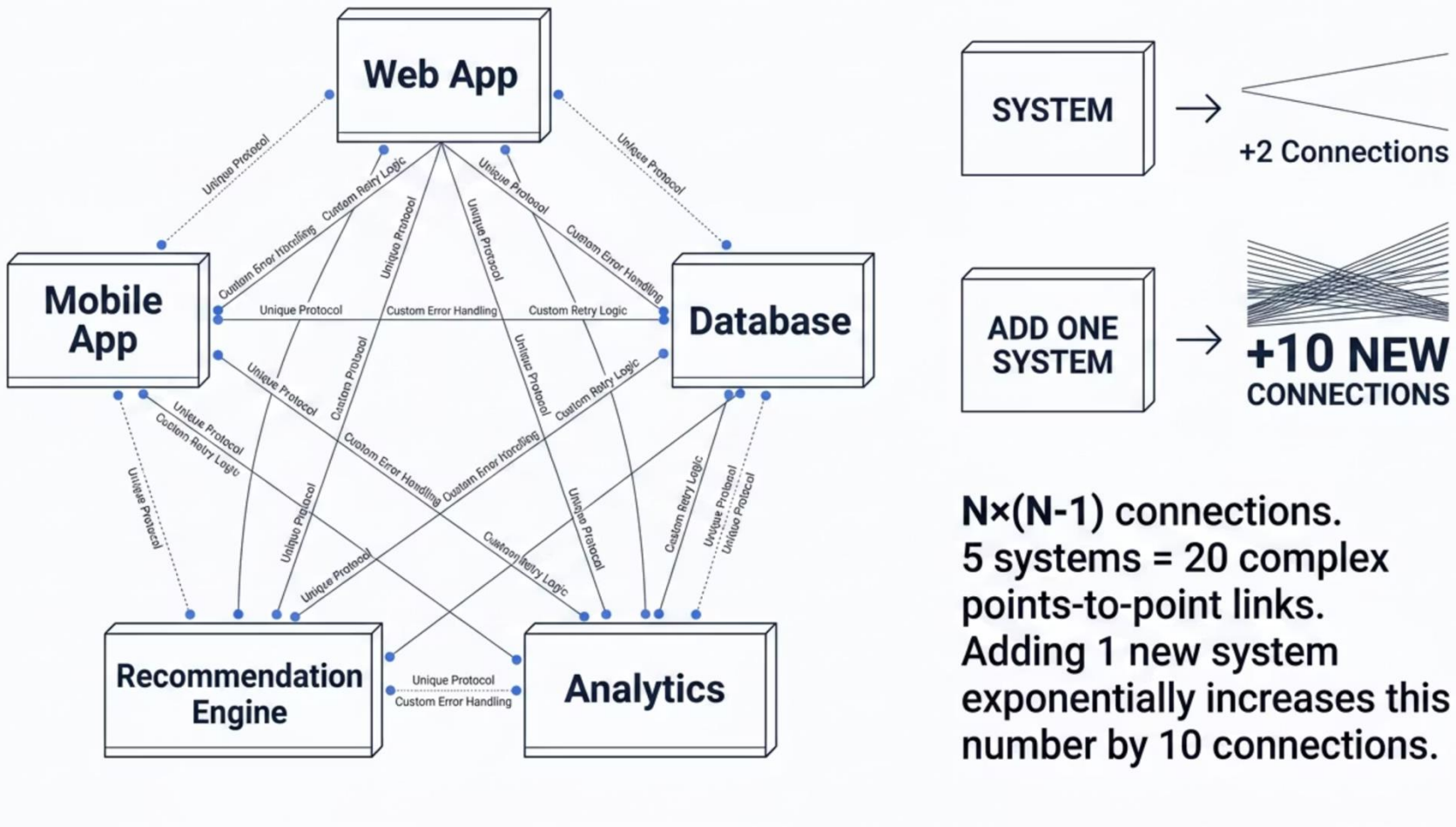
A messaging system is software infrastructure that lets applications communicate by sending and receiving messages — **without needing to be directly connected or even running at the same time.** Instead of Application A calling Application B directly, A sends a message to a broker, and B reads it when it is ready.

💡 **Key Concept — Decoupling:** The most important thing a messaging system does is decouple the sender from the receiver. The sender (producer) does not know who will read the message, when, or how many systems will read it. This separation is what makes large-scale distributed systems possible.



# The Problem Without a Messaging System

Imagine a company with five systems: a web app, mobile app, database, recommendation engine, and analytics platform. Withouta messaging layer, every pair needs a direct connection.




⚠ ⚠ **The Math:** 5 systems = 20 custom integrations. Every integration has its own protocol, error handling, and retry logic. When one system changes its API, every connected system breaks. Adding 1 new system requires 10 new connections.

# Two Classic Messaging Patterns

All messaging systems are built around one of two fundamental patterns. Kafka's design combines elements of both while overcoming their individual limitations.

## Point-to-Point (Queue)

Producer sends to a named queue. **Exactly one consumer** reads and removes the message. Message is deleted after being consumed once.

**Use case:** Task queues, job processing, order fulfilment

**Example systems:** RabbitMQ (AMQP), ActiveMQ

**Limitation:** Only one consumer can process each message.

## Publish / Subscribe (Pub/Sub)

Producer publishes to a topic. **All subscribed consumers** receive a copy. Message is retained until all subscribers receive it.

**Use case:** Event notifications, log broadcasting, analytics feeds

**Example systems:** Apache Kafka, Google Pub/Sub, AWS SNS

**Limitation:** Messages are lost if no subscriber is listening at publish time (in traditional pub/sub).

# Limitations of Traditional Messaging Systems

Traditional brokers like RabbitMQ and ActiveMQ solve the connection problem, but were designed for low-to-medium throughput scenarios. They struggle in five key areas:

### Message Retention

Messages are deleted after consumption. You cannot replay historical data or re-read a message a second consumer missed.

### Throughput Ceiling

Traditional brokers handle thousands of messages per second. Kafka handles **millions**.

### Scalability Model

Adding consumers or topics requires significant reconfiguration. Kafka scales horizontally by adding partitions and brokers.

### Stream Processing

Traditional systems treat messages as isolated tasks, not as a continuous ordered stream of events.

### Durability

Many traditional systems store messages in memory first — broker failure can cause data loss. Kafka writes directly to disk with replication.

> "In comparison to most messaging systems, Kafka has better throughput, built-in partitioning, replication, and fault-tolerance which makes it a good solution for large scale message processing applications." — Apache Kafka Official Docs

SECTION 1.2

# Why Apache Kafka?

## The LinkedIn Problem — 2010

Apache Kafka was not designed in the abstract. It was built to solve a very specific, real-world crisis at LinkedIn — and understanding that story tells you everything about what Kafka was optimized for.

# How Kafka Was Born

1

## 2010 — The Crisis

LinkedIn had hundreds of millions of users. Every page view, search, connection, and job application generated events that dozens of internal systems needed simultaneously.

2

## The Tangled Web

Each system had its own pipeline — custom code, custom protocols, custom retry logic. When one API changed, every connected pipeline broke. Adding any new system meant writing N new pipelines.

3

## The Mission

Jay Kreps, Neha Narkhede, and Jun Rao set out to build a single unified platform: any system could publish to it, any system could subscribe from it — handling billions of events per day with sub-second latency.

4

## 2011 — Open Source

They built Apache Kafka and open-sourced it. The name "Kafka" was chosen because Jay Kreps liked the author Franz Kafka — there is no technical significance to the name.

# What Kafka Is: The Official Definition

Apache Kafka is an **event streaming platform**. According to the official documentation, it combines three key capabilities into a single solution.

### ① Publish & Subscribe

Write and read streams of events — including continuous import/export of data from other systems. Any application can produce or consume.

### ② Store Durably

Store streams of events reliably on disk for as long as you want — from minutes to years — with full replication across brokers.

### ③ Process

Process streams of events as they occur in real time *or* retrospectively — replay history to fix bugs, rebuild state, or train ML models.

> "Kafka combines three key capabilities so you can implement your use cases for event streaming end-to-end with a single battle-tested solution."
> — Getting Started | Apache Kafka

# Kafka's Six Key Advantages

Each advantage below was a deliberate engineering decision made to meet LinkedIn's requirements — not an accidental feature.

## High Throughput

Millions of messages/sec on commodity hardware. Writes are sequential (append-only), faster than random writes. LinkedIn processes **7 trillion messages/day**.

## Horizontal Scalability

Add brokers and partitions to scale capacity. Netflix handles **700+ billion events/day**. No reconfiguration of existing producers or consumers needed.

## Multi-Consumer Independence

Multiple independent consumer groups each read the same topic at their own pace. One group can be 5 minutes behind; another can replay from 3 days ago — neither affects the other.

## Fault Tolerance

Messages are replicated across multiple brokers. If one broker fails, another has a complete copy and takes over automatically — zero data loss (configurable).

## Disk-Based Durability

Every message written to disk immediately. Messages survive broker restarts. Retention is configurable from minutes to years — consumers can re-read anytime.

## Stream + Batch from One Source

The same Kafka topic feeds both real-time processing and batch jobs. No separate pipelines needed for real-time and offline analytics.

# Kafka vs. Traditional Messaging Systems

| Feature | Apache Kafka | RabbitMQ / ActiveMQ |
|---|---|---|
| Message retention | Retained on disk (hours to years) | Deleted after consumer acknowledgement |
| Replay messages | Yes — consumer seeks to any offset | No — once consumed, gone |
| Throughput | Millions of messages/second | Thousands of messages/second |
| Consumer model | Pull-based; consumer controls pace | Push-based; broker pushes to consumer |
| Ordering guarantee | Per-partition ordering guaranteed | Best-effort or queue-level only |
| Horizontal scaling | Add partitions and brokers dynamically | Complex, requires broker clustering config |
| Multiple consumers | Independent consumer groups read same data | Competing consumers split the messages |
| Primary model | Distributed log / event streaming | Traditional message queue / pub-sub broker |

# Real-World Kafka Use Cases

## Website Activity Tracking

LinkedIn's original use case: every page view, search, click, and user action captured in real time.

## Log Aggregation

Collect logs from hundreds of servers and stream them to Elasticsearch, Splunk, or HDFS.

## Stream Processing

Multi-stage pipelines: raw data in → clean/aggregate → enrich → output to target systems.

## Event Sourcing

Store every state change as an immutable event log — reconstruct any past state by replaying events.

## Real-Time Analytics

Power dashboards that update within seconds: fraud detection, IoT sensor analysis, stock prices.

## Payments & Transactions

Process millions of payment events per second with exactly-once delivery guarantees.

SECTION 1.3

# Apache Kafka Basics

Before you can work with Kafka, you must understand its five core building blocks. Together, these concepts form the complete mental model for everything else in this training.

1

## Messages (Events)

The fundamental unit of data

2

## Topics & Partitions

Organization and parallelism

3

## Offsets

Position tracking in the log

4

## Producers & Consumers

Writers and readers of data

5

## Brokers & Clusters

The server infrastructure

# Messages (Events) — The Unit of Data

A message (also called a **record** or **event**) is the fundamental unit of data in Kafka. Everything Kafka does — storing, replicating, processing — operates on messages.

### 📌 Kafka is Schema-Agnostic

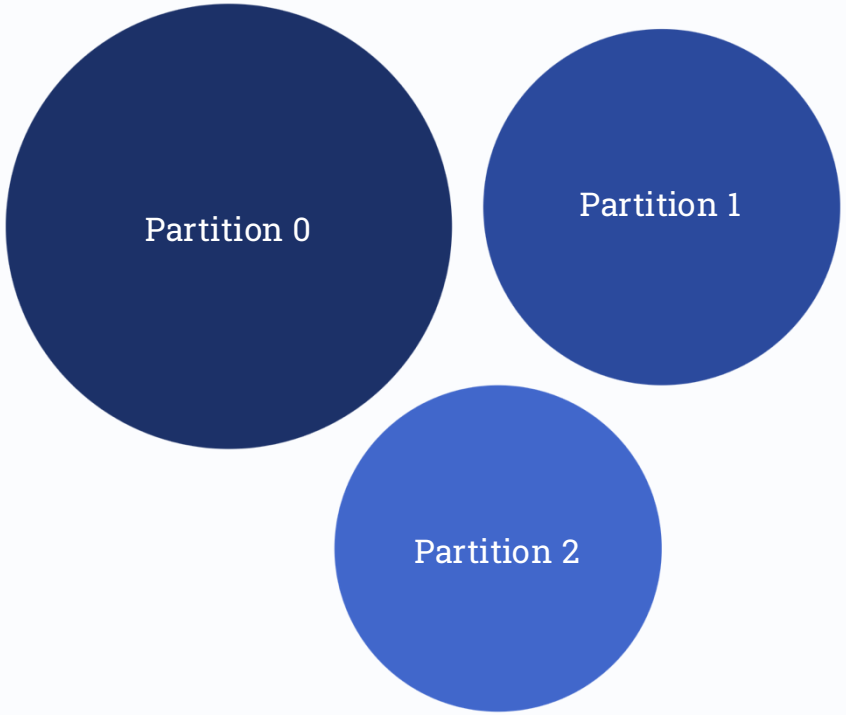
Kafka does not care what format your message value is in — it is just bytes. Schema enforcement (Avro, JSON Schema, Protobuf) is handled by a Schema Registry sitting alongside Kafka, not by Kafka itself. This makes Kafka extremely flexible.

| Property | Description |
| --- | --- |
| Key | Optional bytes. Determines partition via hashing. Critical for ordering. |
| Value | The actual payload — any byte array: JSON, Avro, Protobuf, plain text. |
| Timestamp | Set by producer or broker. Used for retention and stream windowing. |
| Headers | Optional key-value metadata: tracing IDs, routing info, schema references. |
| Offset | Assigned by broker. Monotonically increasing integer, unique per partition. |

**Message Batches:** Kafka groups messages into batches before writing to disk or sending over the network. Batching amortizes network cost — 1,000 messages in one request instead of 1,000 separate requests. Batches are compressed as a unit (gzip, snappy, lz4, zstd), sometimes achieving 10:1 reduction. Tune with `linger.ms` and `batch.size`.

# Topics and Partitions

A **topic** is the primary organizational unit in Kafka — equivalent to a database table or a folder. Every message is published to a topic. Topics are divided into **partitions**, which are the unit of parallelism and scalability.




Each partition is an ordered, append-only log with its own independent offset sequence. Partition count determines maximum consumer parallelism — you can never have more active consumers in a group than there are partitions in a topic.

### Order Within Partition

ORDER IS GUARANTEED within a partition — not across partitions. Use the same message key to guarantee order for related events.

### Append-Only

Partitions are append-only logs — you can only add to the end. You cannot insert, update, or delete individual messages.

### Count = Parallelism

More partitions = more consumers can read in parallel. Partition count CANNOT be decreased — only increased.

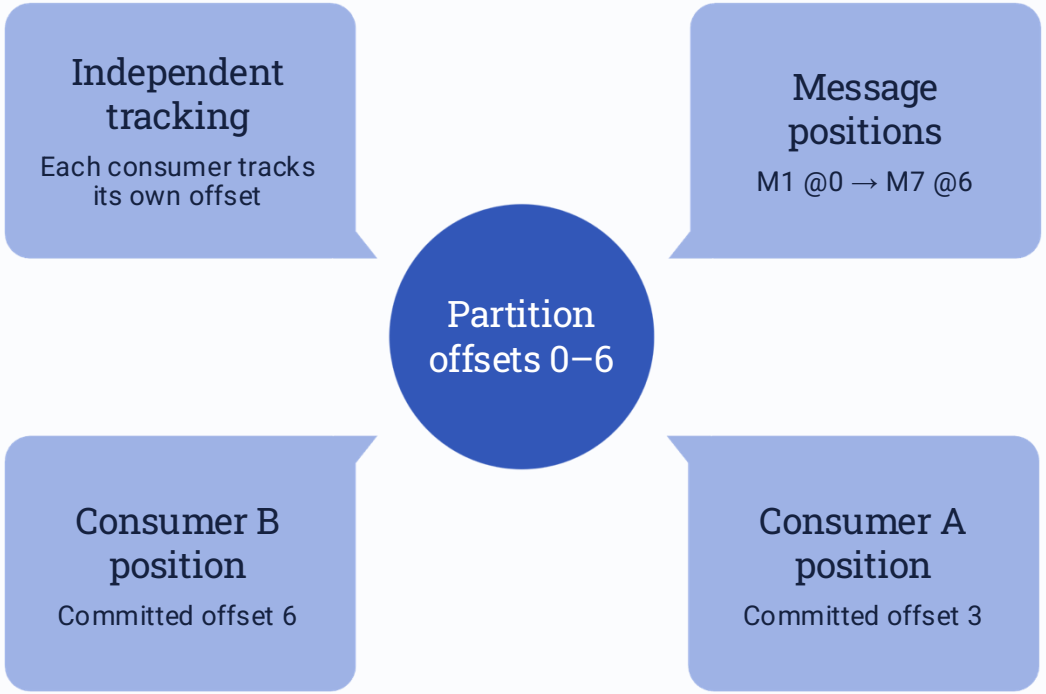
# Topic Configuration Reference

| Config | Default | What It Controls |
|---|---|---|
| `num.partitions` | 1 | Number of partitions. Set at topic creation. Determines max parallelism. |
| `replication.factor` | 1 | Copies of each partition across brokers. Minimum 2 for production; typically 3. |
| `retention.ms` | 604800000 (7 days) | How long messages are kept. Set to -1 for infinite retention. |
| `retention.bytes` | -1 (unlimited) | Max log size per partition before old segments are deleted. |
| `cleanup.policy` | delete | "delete": remove old segments. "compact": keep only the latest message per key. |

Retention is governed by whichever limit triggers first — `retention.ms` or `retention.bytes`. Use `cleanup.policy=compact` for state tables where only the latest value per key matters (e.g., user profile updates).

# Offsets: Kafka's Position System

An **offset** is a sequential integer that Kafka assigns to every message written into a partition. It is Kafka's mechanism for tracking position within the log — and the foundation of consumer fault tolerance.




### Offsets Are Per-Partition

Offset 5 in Partition 0 and Offset 5 in Partition 1 are completely different messages. There is no global offset across partitions.

### Consumers Commit Offsets

A consumer tells Kafka "I have successfully processed up to offset N." Commits are stored in the internal `__consumer_offsets` topic.

### Offsets Are Immutable

Once assigned, an offset never changes. The message at Offset 3 in Partition 0 will always be the same message.

### Replay Is Possible

Any consumer can seek backwards to an earlier offset and re-read messages. This is impossible in traditional queues.

# Producers: Writing Messages to Kafka

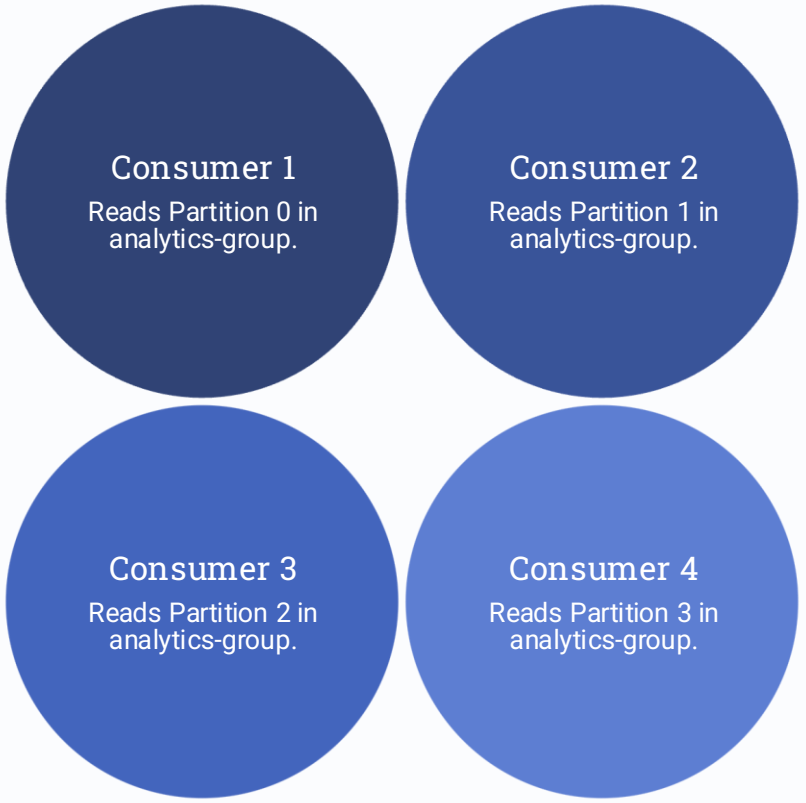
A **producer** is any application that writes (publishes) messages to a Kafka topic. Producers are responsible for choosing which topic and partition a message goes to.


| Config | Options | What It Controls |
|---|---|---|
| `acks` | 0 / 1 / all | 0=fire-and-forget; 1=leader confirms; all=full replication, no loss |
| `batch.size` | 16384 bytes | Max bytes to batch before sending. Larger = more throughput. |
| `linger.ms` | 0 ms | Wait time to fill a batch. 5–10ms boosts throughput significantly. |
| `compression.type` | none/gzip/snappy/lz4/zstd | Compression algorithm for batches. Snappy is a good default. |
| `enable.idempotence` | true/false | Exactly-once delivery — prevents duplicate writes on retry. |
| `retries` | MAX_INT | Retry count on transient failures before giving up. |

# Consumers and Consumer Groups

A **consumer** reads (subscribes to) messages from Kafka topics using a **pull model** — requesting messages at their own pace rather than having the broker push to them. Every consumer belongs to a **consumer group**.

**Why Pull vs. Push Matters:** In a push-based system (like RabbitMQ), the broker controls the rate messages arrive. If consumers are slow, they get overwhelmed. In Kafka's pull model, each consumer reads at its own pace — a slow consumer just falls behind without affecting others or the broker.




The consumer group is Kafka's mechanism for dividing work among multiple consumers while ensuring each message is processed **exactly once within the group**. The key rule: each partition is assigned to exactly one consumer within a group at any time.

# Consumer Group Partition Assignment Scenarios

| Scenario | Partition Assignment | Effect |
| --- | --- | --- |
| Consumers < Partitions (2 consumers, 4 partitions) | Consumer 1: P0+P1 · Consumer 2: P2+P3 | Works fine. Each consumer handles more partitions. |
| Consumers = Partitions (4 consumers, 4 partitions) | C1:P0 · C2:P1 · C3:P2 · C4:P3 | Ideal. Maximum parallelism — 1 partition per consumer. |
| Consumers > Partitions (6 consumers, 4 partitions) | 4 consumers active · 2 consumers idle | Two consumers receive no partitions. Adding more than partitions buys nothing. |
| Consumer fails (C2 crashes) | Kafka rebalances — remaining consumers absorb C2's partitions | Kafka detects failure via heartbeat timeout and rebalances automatically (typically within 10 seconds). |

⚠ ⚠ **Consumer Group Rebalancing:** When consumers join, leave, or crash, Kafka triggers a rebalance — all consumers stop consuming while Kafka reassigns partitions. Kafka 2.4+ introduced **Incremental Cooperative Rebalancing**, which allows consumers to keep their current partitions during rebalance, dramatically reducing disruption. Kafka 4.x (KRaft) further improves rebalance performance.

# Brokers and Clusters

A **broker** is a single Kafka server. It receives messages from producers, stores them on disk, and serves them to consumers. A **cluster** is a group of brokers working together — you will almost never run Kafka as a single broker in production.

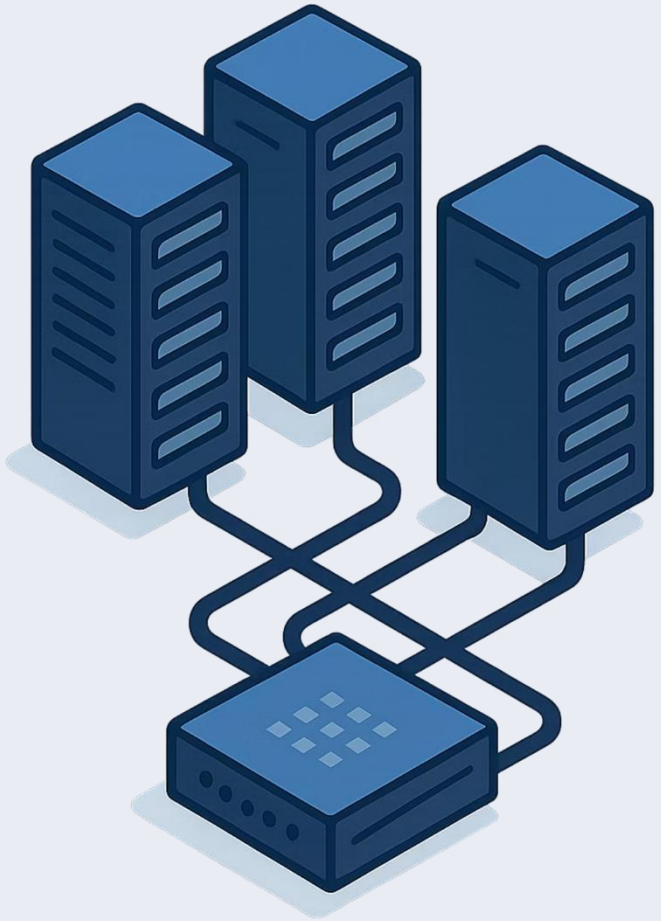
| Broker Role | Description |
| --- | --- |
| Leader | Handles all reads and writes for a partition. Only one per partition at any time. |
| Follower | Replicates data from the Leader. Takes over automatically if the Leader fails. |
| ISR | In-Sync Replica — a follower fully caught up with the Leader. Only ISR members can become the new Leader. |
| Controller | Manages cluster metadata and partition leadership. Elected via KRaft in Kafka 4.x. |
| Bootstrap Broker | Listed in `bootstrap.servers`. Clients connect here first to discover the full cluster. |

### KRaft vs. Zookeeper

**Kafka 0.x–3.x:** Used Apache Zookeeper for cluster coordination — a separate process on port 2181.

**Kafka 4.x (this guide):** Zookeeper is REMOVED entirely. Cluster metadata is managed using the Raft consensus protocol built into Kafka itself.

**KRaft benefits:** Simpler operations, faster controller failover (milliseconds vs. seconds), supports millions of partitions vs. ~200K with Zookeeper.



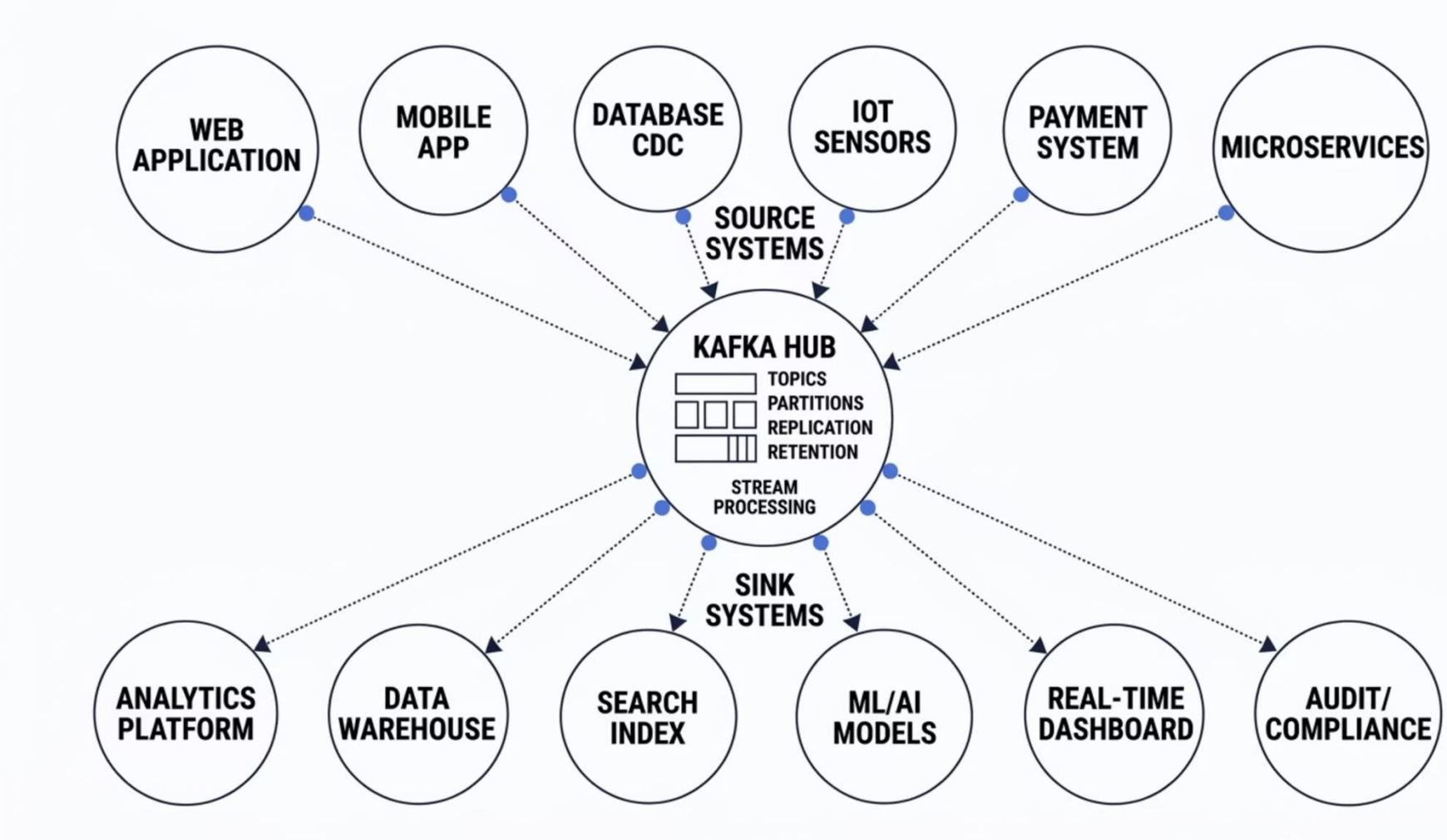
SECTION 1.4

# The Data Ecosystem

In modern organizations, data is produced by dozens of systems — web apps, mobile apps, databases, IoT sensors, payment processors — and consumed by analytics platforms, data warehouses, search indexes, dashboards, and more. Without a central data platform, every source must connect to every sink, creating an **N×M integration problem at enterprise scale.**

Apache Kafka solves this by acting as the **central data hub** for the entire organization. Each source system needs exactly one connection — to Kafka. Each sink system needs exactly one connection — to Kafka. Adding a new source or sink requires zero changes to existing systems.

# Kafka as the Central Nervous System




With Kafka at the center, this is the architecture that LinkedIn, Netflix, Uber, Airbnb, and virtually every major data-intensive company uses. Any new system connects to Kafka once — not to every other system individually.

# Kafka Connect: The Integration Layer

**Kafka Connect** is a framework included with Kafka that provides pre-built, configurable connectors for common source and sink systems. Instead of writing custom producer/consumer code, you use a connector configuration file.

## Connector Types

**Source Connector:** External system → Kafka

**Sink Connector:** Kafka → External system

Example: Stream every row inserted into a MySQL table directly into a Kafka topic in real time — no code required.

## Key Benefits

- **No code required** — configure via JSON files
- **Fault tolerant and scalable** — runs as a cluster of workers, automatically rebalances on failure
- **Exactly-once semantics** — when used with transactional Kafka topics
- **200+ pre-built connectors** — databases, cloud storage, search engines, and more

# Real-World Data Ecosystem Examples


## LinkedIn — Kafka's Birthplace

Central event bus for **7 trillion+ messages/day**: activity tracking, recommendations, metrics, job alerts, and newsfeed ranking — the original use case that created Kafka.

## Netflix

**700+ billion events/day**: stream processing for personalization, playback analytics, A/B test analysis, and operational dashboards across their global streaming platform.

## Uber

Real-time surge pricing, driver location tracking, fraud detection, and ETL pipelines across **4,000+ Kafka topics** handling millions of rides daily.

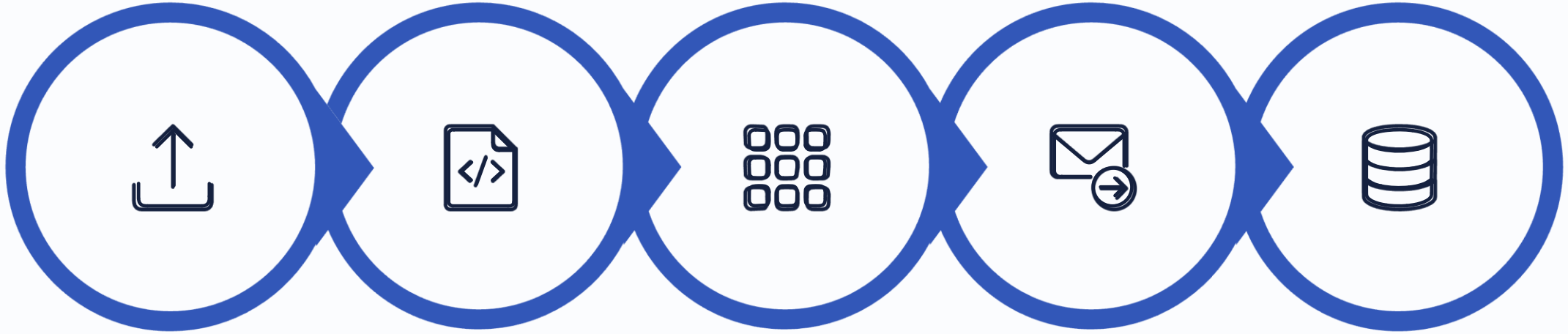
## Goldman Sachs

High-frequency trade event processing, regulatory reporting, risk analysis, and market data distribution — all powered by Kafka's exactly-once guarantees.

SECTION 1.5

# The Complete Kafka Workflow

Now that you understand each component, let's trace the complete lifecycle of a single message — from the moment an application creates an event to the moment a consumer processes it. This 8-step workflow is the foundation you will use throughout all remaining training modules.



Produce Serialize Partition Batch & Send Broker Store

# Message Lifecycle — Steps 1–4

## 1 Producer Creates Message

The application (e.g., a checkout service) creates a `ProducerRecord` specifying: topic name, an optional key (e.g., user ID), and a value (e.g., JSON order data). Optionally sets timestamp and headers.

## 2 Serialization

The Producer API serializes the key and value to bytes using the configured serializers (e.g., `StringSerializer`, `JsonSerializer`, `AvroSerializer`). Kafka only stores bytes — serialization is the application's responsibility.

## 3 Partition Selection

The Partitioner determines which partition receives the message: If key != null → `partition = hash(key) % numPartitions`. Same key always → same partition. If key == null → round-robin across all partitions.

## 4 Batching and Sending

The message is added to an in-memory batch for its partition. The batch sends when `batch.size` bytes is reached OR `linger.ms` milliseconds elapse — whichever comes first. The batch may be compressed before sending.

# Message Lifecycle — Steps 5–8

## 5 Broker Receives and Assigns Offset

The leader broker for that partition receives the batch. For each message, it assigns the next sequential offset (0, 1, 2...). The message is written to the partition's log file on disk (append-only) and held in the OS page cache for fast reads.

## 6 Replication to Follower Brokers

If `replication.factor > 1`, follower brokers pull new messages from the leader and replicate them to their own disk. Once all in-sync replicas (ISR) have acknowledged (if `acks=all`), the leader sends an acknowledgement back to the producer.

## 7 Consumer Polls and Fetches

The consumer application calls `poll()` in a loop. Kafka delivers available messages from the consumer's assigned partitions starting from its committed offset. The consumer deserializes the bytes and processes them (business logic).

## 8 Offset Commit

After processing, the consumer commits its offset back to Kafka (stored in `__consumer_offsets`). This is the consumer's "bookmark." If the consumer crashes and restarts, it resumes from this committed offset — not from the beginning.

# Retention Policy: Messages Outlive Consumers

Unlike traditional queues that delete messages after consumption, Kafka keeps messages for a configurable retention period. This is one of Kafka's most powerful and distinctive features.

| Config | Default |
| --- | --- |
| `log.retention.hours` | 168 (7 days) |
| `log.retention.bytes` | -1 (unlimited) |
| `log.segment.bytes` | 1 GB |
| `log.cleanup.policy` | delete |

## Why Retention Transforms Architecture

- **Decoupled consumers:** A consumer offline for 6 days can catch up — messages are still there.
- **Replay capability:** Reprocess historical events to fix bugs, rebuild state, or train ML models.
- **Multi-speed consumers:** A real-time system and a daily batch job both read the same topic at completely different rates.
- **Audit trail:** Immutable history of all events for compliance (e.g., payment transactions).

KNOWLEDGE CHECK

# Module 1 — Knowledge Check

Test your understanding before moving to Module 2. Try to answer each question before checking the answer key at the bottom.

1

## Q1 — What is a Topic?

Which of the following best describes an Apache Kafka "topic"?

A) A single message containing a key and a value
B) A named, ordered, and partitioned log of messages
C) A configuration file that defines consumer behaviour
D) A broker process that manages cluster metadata

2

## Q2 — Consumer Groups

A Kafka topic has 4 partitions and a consumer group has 6 consumers. How many consumers will receive messages?

A) 6 — all consumers receive a copy
B) 4 — only 4 consumers are active; 2 are idle
C) 1 — only the group leader processes messages
D) 2 — consumers alternate

3

## Q3 — Key-Based Partitioning

Producer A sends messages with key='user-42' to a topic with 10 partitions. Where will all of these messages land?

A) Randomly distributed across all 10 partitions
B) Only in Partition 0
C) Always in the same partition (determined by hash of 'user-42')
D) In whichever partition has the most available space

4

## Q4 — Offset After Crash

A consumer reads offset 50 in Partition 3 and commits. The consumer crashes. When it restarts, from which offset does it resume?

A) Offset 0 — beginning of the partition
B) Offset 50 — the committed checkpoint
C) The latest offset
D) It cannot resume — messages are lost

5

## Q5 — KRaft Mode

Which Kafka mode (default in Kafka 4.x) eliminates the dependency on Apache Zookeeper?

A) SASL mode
B) ISR mode
C) KRaft mode (Kafka Raft)
D) Mirror mode

# Knowledge Check — Short Answer Questions

### Q6 — Messaging Patterns

Explain the difference between a point-to-point (queue) messaging pattern and a publish/subscribe pattern. Give one real-world example of each.

### Q7 — The N×(N-1) Problem

What is the N×(N-1) problem? How does introducing a central messaging system like Kafka solve it? What happens when you add a new system to the architecture?

### Q8 — Consumer Group Exclusivity

Why can two consumers in the same consumer group NEVER read from the same partition simultaneously? What are the consequences and recovery mechanism if a consumer crashes mid-processing?

### Q9 — Producer acks Setting

A developer sets `acks=1` on their Kafka producer. What does this mean, and what data loss scenario does it leave open compared to `acks=all`? When would you choose one over the other?

**Answer Key (Multiple Choice):** Q1: B · Q2: B · Q3: C · Q4: B · Q5: C
Short answer questions are discussion-based — review against sections 1.1 through 1.5 of this module.